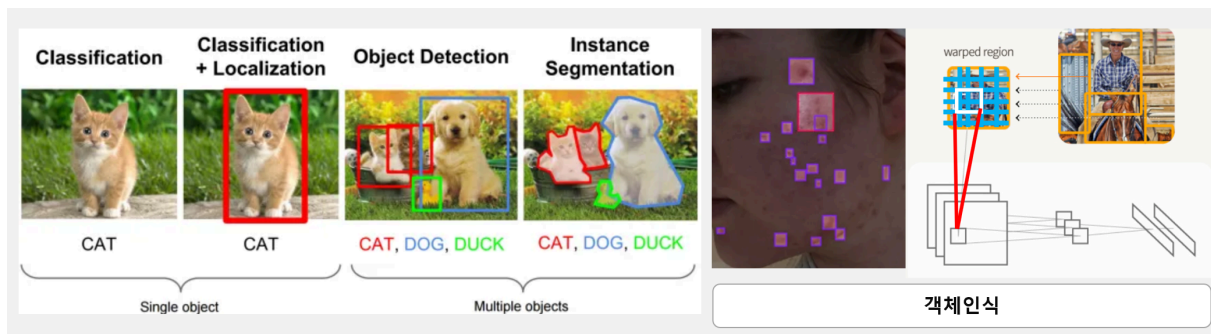


1. 객체인식 개요

○ 학습목표

- 분류와 객체인식의 차이점을 확인, 객체인식의 히스토리 이해
- R-CNN의 원리만 이해
- 객체인식의 용어 알기



1 이미지분류 와 객체인식의 차이

○ 이미지분류

CNN 분류(Classification)는 “사진 1장 = 라벨 1개” 즉, 이미지 전체를 하나의 종류로 판단하는 모델.

✓ CNN Classification 사고방식

“이 사진 전체를 보면 털 + 귀 모양 + 눈 + 얼굴 패턴 → 고양이네!”

즉, 전체 그림을 단일 라벨로 매핑.

예시: 사진을 넣으면 → “개다”, “고양이다”, “사람이다” *하나만* 출력.

[입력 이미지] → [CNN] → "고양이"

✓ 특징

- 이미지에 **무엇이 있는지**만 말해줌
- 어디에 있는지(위치)는 절대 모름
- 이미지 전체를 하나의 큰 덩어리로 봄

○ 객체인식(Object Detection)

👉 "사진 안에 여러 객체가 있을 때, 각각을 찾아서 + 위치까지 표시"

즉, 사진을 여러위치로 스캔하면서 (이미지를 슬라이딩)하며 여러 위치를 탐색

- **무엇인지(Class)**
- **어디 있는지(Bounding Box)**

둘 다 알려주는 모델.

[입력 이미지] → [Object Detector]

→ 사람(100,120,300,450)

→ 개(20,50,140,250)

✓ 특징

- 이미지 속 **여러 객체를 동시에** 찾음
- 각 객체의 위치(박스)를 예측
- 분류보다 훨씬 난이도가 높음

✓ 객체인식이 분류보다 어려운 이유

- 위치 정보 추적 필요
 - 객체 개수가 이미지마다 다름
 - 크기·각도가 달라도 찾아야 함
 - 겹쳐 있을 수도 있음
- 분류보다 훨씬 복잡한 구조 필요 (SSD, YOLO 등장 이유)

○ 비교

참고) 객체인식 **SSD(Single Shot Detector)** → CNN에 '박스 예측 헤드'를 여러 개 붙인 모델

먼저: CNN 분류(Classification)는 "무엇인지만" 맞춘다

📌 예시: 고양이 vs 강아지

- 입력: 이미지 (224×224×3)
- CNN 특징추출 → FC층 → softmax
- 출력: 1개 클래스
 - 예: [고양이 0.9, 강아지 0.1]

즉, 이미지 전체에 단 하나의 라벨만 달아준다.

분류

이미지

↓
CNN 특징 추출 (Conv → Pool → Conv ...)

↓
Feature Map (예: 7×7×256)

↓
각 격자셀(7×7)이 아래 2개를 예측:

1) 객체가 있는지 (objectness)

2) 박스 위치 (x,y,w,h) + 클래스

↓
NMS(겹치는 박스 정리)

↓
최종 객체 박스 출력

객체인식

항목	CNN Classification	Object Detection
목적	이미지 전체 클래스 판별	여러 객체 + 위치 찾기
출력	1개의 라벨	라벨 + 박스 여러 개
난이도	낮음	높음
사용 예	고양이/개 분류	CCTV 사람 탐지, 차량 번호판, 공장 불량 감지
시점	전체 그림만 분석	그림 속 "지역별" 분석

2 객체인식(Object Detection) History

① 전통적 방법 (딥러닝 이전) — 2000년대 HOG + SVM, Haar + Adaboost(비올라-존스)

- 규칙 기반(feature hand-craft)
- 특징(HOG, Haar)을 사람이 정의

- 속도는 빠르지만 **정확도 낮고**, 복잡한 이미지에서 잘 안 됨 → 스마트폰 부품 결함 검출엔 부적합

② R-CNN 시대 (Region Proposal 기반) — 2014~2015 딥러닝 기반 객체 인식의 시작

모델	연도	주요 특징	속도
R-CNN	2014	Selective Search로 영역 후보 2000개 뽑고, 각 영역을 CNN에 넣어 classification	너무 느림 (초당 1~2fps)
Fast R-CNN	2015	전체 이미지 한 번만 CNN, ROI Pooling 사용	R-CNN보다 10~20배 속도 ↑
Faster R-CNN	2015	Selective Search 제거 → RPN(Region Proposal Network) 사용, 속도와 정확도 크게 향상 → 지금도 정확도는 매우 높음 → 하지만 속도 느림 , 실시간 딥러닝 불가	Fast R-CNN보다 향상

③ YOLO 시대 (실시간 객체 인식) — 2016~현재 **["한 번에(One-Shot)" 예측 → 실시간 가능]**

- 이미지 → CNN **1번 통과**
- 동시에 **객체 위치 + 클래스 예측**
- 매우 빠름(30~300fps)
- 정확도도 최근엔 최고수준(YOLOv5, v8, v11), YOLOv8/v11도 중요 부분에 Transformer를 쓰기 시작

특징

YOLO = 속도 최강(실시간), 구조 단순, 최신 성능도 매우 좋음

→ 스마트폰 PCB 검사, 작업자 안전감지, 생산라인 실시간 이미지 분석에 적합

④ SSD (Single Shot Detector) — 2016 YOLO와 비슷한 **"1회 추론(one-stage)"** 방식

- 여러 크기의 feature map에서 예측 → 작은 객체에 강함
- 모바일에서도 빠름 (MobileNet-SSD)

- 구조가 심플하고 가벼움

특징

YOLO보다 살짝 정확도 낮고, Faster RCNN보다 속도 빠름.

SO, 정확도·속도 중간 포지션.

⑤ RetinaNet — 2017

- One-stage(YOLO·SSD)의 속도
- Two-stage(FasterRCNN)의 정확도
- Focal Loss 로 클래스 불균형 문제 해결

⑥ Transformer 기반 객체 인식 — 2020~현재

DETR / DINO / RT-DETR

- CNN 아예 제거하고 Transformer 기반
- Post-processing(NMS) 필요 없음
- 최근 SOTA 최고성능 → 하지만 모델 크고 학습이 어렵고 느림

모델	속도	정확도	구조	특징
R-CNN	느림	중간	2단계	최초 CNN 기반 검출
Fast/Faster R-CNN	중간	높음	2단계	정확도 최고
YOLO	빠름	높음	1단계	실시간 검출, 최신 SOTA
SSD	빠름	중간	1단계	가볍고 모바일 친화
RetinaNet	빠름	높음	1단계	Focal Loss
DETR / DINO	느림	최고	Transformer	차세대 방식

- 예: 제조공정에서의 객체인

목적	추천 모델	이유
실시간 생산라인 모니터링	YOLOv8/YOLOv11	정확도·속도 최강

목적	추천 모델	이유
부품 작은 불량(마이크로 결함)	SSD or YOLOv8 with high-resolution	작은 객체 검출 적합
고정밀 보고용 (속도 필요 X)	Faster R-CNN or DETR	최고 정확도

3 CNN계열의 객체인식 - RCNN의 **Selective Search**

1. 객체인식-초간단RCNN.ipynb

○ 객체인식에 사용되는 알고리즘 미리 알기: **SLIC(Superpixel Linear Iterative Clustering)**

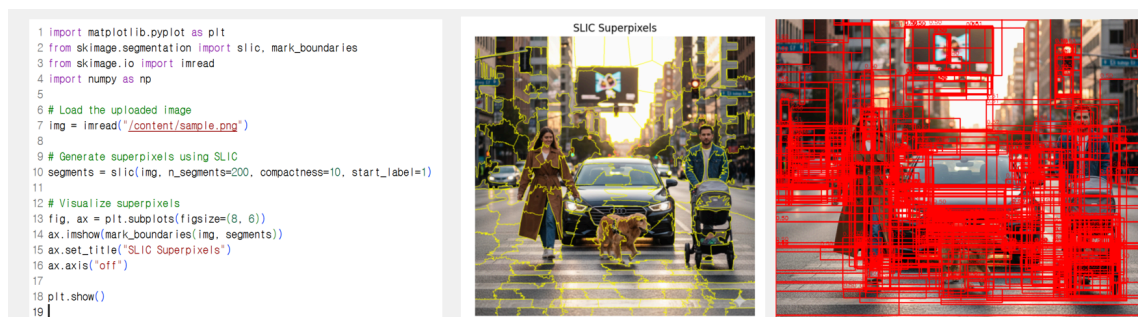
이미지를 “비슷한 색 + 가까운 위치” 기준으로 작은 덩어리(superpixel)로 나누는 알고리즘.

즉, 사진을 ‘작은 영역들’로 미리 잘라주는 기술

객체가 있는 영역을 찾기 위해서는 너무 작은 픽셀 단위 말고, 의미 있는 조각 (**superpixel**) 이 필요함.

SLIC 이미지가 있다고 하면 → 비슷한 색/밝기/텍스처끼리 묶어서 조금 큰 단위로 나눠 주는 느낌

파란 하늘 부분은 하나의 덩어리, 녹색 잔디는 또 하나의 덩어리 , 회색 도로는 하나의 덩어리, 이런 식으로 구역을 나눔.



○ **Selective Search** '1.객체인식-초간단RCNN.ipynb'

- R-CNN 스타일 객체 후보 박스 생성기 대표모듈 :
`cv2.ximgproc.segmentation.createSelectiveSearchSegmentation()`
- 현대에서는 잘 사용되지 않으나, CPU기반의 임베디드에서는 사용될수도 있음. →
현재는 속도: YOLOv8 200~300배 빠름 / 정확도: 현대 모델이 압도적으로 우수

- **Selective Search: 객체 후보 영역(Region Proposals)** 을 빠르게 찾아주는 알고리즘, superpixel들을 합쳐가며 객체 후보 영역을 만드는 알고리즘

딥러닝 이전 시대에서 '어디에 객체가 있을까?' 를 정답 박스 없이 먼저 찾기 위해 사용, 즉 "먼저 후보 지역을 뽑고 → CNN으로 분류" 하는 초창기 방식

이미지를 여러 색·텍스처·크기·형태 기반으로 비슷한 영역끼리 묶어가며 "객체일 가능성이 높은 후보 박스들"을 만들어주는 알고리즘으로, R-CNN(2014)에서 처음 크게 사용되면서 객체 인식(Object Detection)의 핵심 전처리 기술이 됨

- 이미지를 작은 조각(superpixel) 으로 분할 → SLIC 같은 방법으로 비슷한 색/텍스처 영역을 나눔
- 비슷한 조각끼리 점점 합침 (계층적 병합) → 색, 질감, 크기 비율, 경계(edge) 등을 기준으로 유사한 영역을 점점 묶음
- 병합 과정에서 '가능한 모든 영역'이 후보 박스가 됨 → 결과적으로 수백~수천 개의 bounding box가 생성됨

장점

- 딥러닝 없이도 후보 박스를 잘 찾아줌
- 모든 위치를 슬라이딩 윈도우로 훑는 것보다 훨씬 효율적
- R-CNN, Fast R-CNN 시대의 표준 알고리즘

✕ 단점

- 느림 (CPU 기반, 계산량 많음)
- 후보 영역이 많아 학습/추론 속도가 느림
- YOLO/SSD처럼 end-to-end 학습 방식과 비교하면 오래된 방식

모델	Selective Search 사용 여부
R-CNN	✓ 사용함
Fast R-CNN	✓ 여전히 사용함
Faster R-CNN	✗ 사용 안함 (Region Proposal Network로 대체)
YOLO/SSD/RetinaNet	✗ 필요 없음 (One-stage 방식)

Step1. 초기 영역 생성

- Graph-based segmentation 같은 방법으로 이미지를 작은 superpixel들로 분할
- 과분할(over-segmentation)을 의도적으로 해서 객체가 여러 조각으로 나뉘게 함

Step2. 유사도 계산

인접한 superpixel 쌍마다 4가지 유사도를 계산합니다:

- **색상 유사도:** 색상 히스토그램 간 거리 (색이 비슷한가?)
- **질감 유사도:** SIFT 같은 특징의 히스토그램 비교 (표면 질감이 비슷한가?)
- **크기 유사도:** 작은 영역끼리 먼저 합치도록 유도 (균형있게 합쳐지도록)
- **채움 유사도:** 합쳤을 때 빈 공간이 적을수록 높음 (서로 잘 맞물리는가?)

Step3. 반복적 병합

while 남은 영역이 있으면:

1. 가장 유사도가 높은 인접 영역 쌍 찾기
2. 두 영역 합치기
3. 합쳐진 새 영역의 bounding box 저장 → 객체 후보!
4. 새 영역과 이웃들 간의 유사도 다시 계산

다양성 확보 전략

같은 이미지를 여러 방식으로 처리해서 다양한 후보를 얻습니다:

- **다양한 색공간:** RGB, HSV, Lab 등
- **다양한 유사도 조합:** 색상만, 질감만, 색상+질감+크기 등
- **다양한 초기 분할:** 파라미터를 바꿔가며 여러 버전

이렇게 해서 한 이미지당 약 2,000개의 region proposal을 생성합니다.

왜 효과적인가?

- **계층적 구조:** 작은 부품 → 중간 크기 객체 → 큰 객체까지 자연스럽게 포착
- **다중 전략:** 색깔이 비슷한 객체도, 질감이 비슷한 객체도 모두 잡아냄
- **효율성:** 슬라이딩 윈도우(수십만 개)보다 훨씬 적은 후보(~2000개)로 높은 재현율

예를 들어, 빨간 사과를 찾는다면: 빨간 superpixel들이 색상 유사도로 먼저 합쳐지고, 초록 잎사귀는 별도로 합쳐지다가, 나중에 "사과+잎" 전체가 하나의 후보가 되는 식임

Selective Search는 현대에서는 사용되지 않으나 객체 검출 발전사에서 중요한 역할을 함.

- **R-CNN (2014)의 핵심 구성요소로 딥러닝 객체 검출 시대를 열었음 / "학습 가능한 region proposal"의 필요성을 보여줌 / 이후 Faster R-CNN 같은 혁신으로 이어지는 계기**
- **Selective Search 문제점: 속도 문제 / End-to-End 학습 불가 / 정확도 한계**
 - CPU 기반으로 동작 (GPU 활용 불가) / 이미지 하나당 2~3초 소요 / 실시간 처리가 불가능
 - 수작업으로 설계한 휴리스틱에 의존 / 딥러닝 모델과 함께 학습되지 않음 / 최적화가 분리되어 있어 성능 한계
 - 고정된 규칙 기반이라 복잡한 상황에 약함 / 겹친 객체, 복잡한 배경에서 성능 저하
- **현대적 방법**
 - **Region Proposal Network (RPN) - Faster R-CNN (2015)**
 - 딥러닝 기반으로 region proposal 생성 / GPU에서 빠르게 동작 / 전체 네트워크와 함께 end-to-end 학습
 - **Anchor-free 방식 - YOLO, RetinaNet**
 - Region proposal 단계 자체를 없앴 / 직접 객체 위치와 클래스 예측 / 훨씬 더 빠르고 간단
 - **Transformer 기반 - DETR (2020~)**
 - Set prediction으로 객체 검출을 재정의 / NMS, anchor 같은 수작업 설계 완전 제거

4 미리보기: SSD = Single Shot Detector

- **Single Shot**
 - 한 번(=single shot) CNN을 통과시키는 forward로 곧바로 모든 박스 + 클래스를 예측

- 예전 R-CNN 계열은
"1단계: 후보 박스 뽑기 → 2단계: CNN 분류"처럼 **2스텝**이었음
- SSD/YOLO는 "한 방에" 끝내서 single shot

- **MultiBox**

- 각 위치마다 **여러 개의 기본 박스(default box, anchor)**
(다른 크기·비율의 박스들)을 깔아두고
그 박스들을 조금씩 수정(offset)해서 최종 박스로 사용
- 그래서 "여러 박스(MultiBox)" 개념이 붙음

즉, "한 번에 여러 기본 박스를 잡아서 객체를 찾는 CNN" → SSD

- **백본 CNN** → 이미지 특징 추출
- **추가 CNN 레이어** → 더 작은 feature map 생성
- 각 feature map의 각 위치마다
 - **박스 좌표 (4개 regress)**
 - **클래스 점수 (num_classes개)**
 를 동시에 예측 → 이것을 **Single Shot** 이라고 함

5 용어미리알기

- 어노테이션(Annotation): 인공지능(AI) 모델을 학습시키기 위해 **데이터(이미지, 텍스트, 비디오 등)에 메타데이터(설명 정보 또는 라벨)를 추가하는 작업**
- 객체인식 용어리스트

번호	용어	의미
1	Bounding Box (BBox)	객체를 둘러싸는 사각형 영역
2	Ground Truth (GT)	사람이 직접 표시한 정답 박스
3	Region Proposal	객체일 가능성이 있는 후보 박스
4	Anchor Box	사전 정의된 박스 크기/비율(One-stage 모델에서 사용)
5	IoU (Intersection over Union)	예측 박스와 정답 박스의 겹치는 비율
6	Confidence Score	객체가 있을 확률(모델 출력)
7	Class Probability	해당 클래스일 확률(softmax 결과)

번호	용어	의미
8	NMS (Non-Max Suppression)	겹치는 박스를 제거하는 후처리 단계
9	mAP (mean Average Precision)	객체 탐지 모델의 대표적인 성능 평가 지표
10	AP50 / AP75	IoU 기준값(50%, 75%)에 따른 AP

